RFID Based Cashless Multipurpose Wallet

Abhishek Gautam
dept. of Electrical Engg.
ag373@snu.edu.in
2410110396

Bhavy Garg
dept. of Electrical Engg.
bg360@snu.edu.in
2410110416

Ishan Chudasama
dept. of Electrical Engg.
ic929@snu.edu.in
2410110430

Naveen Kumaar
dept. of Electrical Engg.
nc688@snu.edu.in
2410110448

Pratham Garg
dept. of Electrical Engg.
pg816@snu.edu.in
2410110460

Sankarasubrammanian Anantharaman
dept. of Electrical Engg.
sa753@snu.edu.in
2410110470

Saumya Agarwal
dept. of Electrical Engg.
sa787@snu.edu.in
2410110473

***Abstract*—A secure, contactless RFID-based digital wallet system using STM32 to enable fast campus payments with real-time display and multi-vendor support**

I. Introduction

The RFID Wallet System is a smart, cashless payment solution designed for campus use. It allows students to make secure payments by simply tapping their RFID cards, while vendors use a separate RFID card to select items.

The system uses an STM32 microcontroller, MFRC522 RFID reader, 16×2 LCD, and a buzzer to provide fast, contactless transactions with real-time display and audio feedback. Student cards can also show balance, add money, and view transaction history.

With automatic timeouts, low-balance alerts, and multi-vendor support, this project provides a reliable, modern, and efficient digital wallet experience for educational campuses.

A. Motivation & Inspiration

The motivation for this project comes from widely used, efficient card-based systems that simplify daily transactions and access. The Metro Card demonstrates how tap-and-go technology can handle large crowds seamlessly, providing fast, contactless access in high-throughput environments like public transport. This inspired the need for a similarly quick and reliable system for campus payments. The Library Card model shows how a single ID can store essential user information and privileges, making identification simple and convenient—an idea mirrored in student RFID cards that store balance and transaction data. Finally, Hong Kong's Octopus Card highlights the power of an integrated, multi-purpose smart card that works across transport, retail, and dining. Its versatility and ease of use inspired the multi-vendor support and cashless functionality in this project. Together, these real-world systems motivated the creation of an efficient, secure, and unified RFID wallet system for educational campuses.

B. Key Features

The system offers several key features that make campus transactions fast, secure, and user-friendly. Cashless payment allows students to pay instantly by tapping their RFID cards, eliminating the need for physical money. The student menu system lets users check balance, add funds, and view basic options on the LCD. Automatic timeouts ensure the system resets safely if left inactive. With multi-vendor support, the same card can be used at different campus shops or cafeterias. Finally, low-balance alerts notify students when funds are running low, helping them avoid failed transactions and ensuring smooth day-to-day usage.

II. Techincal Specifications

A. Core Components Used

- **STM32F303RET6 NUCLEO** A Family of 32-bit microcontrollers (MCUs) by STMicroelectronics based on the Arm Cortex-M core architecture.
- **MFRC522 RFID** MFRC522 is a high-frequency RFID reader module used to read and authenticate 13.56 MHz contactless cards.
- **LCD 16x2** A 16x2 LCD display is designed to show 16 characters on each of its two lines.
- **BUZZER** The buzzer provides audio feedback for actions like card detection, payments, and errors.

B. Pin Configuration

TABLE I: 16×2 LCD (4-bit Mode) → STM32 (GPIO) Interfacing

LCD Pin	**STM32 Pin**	**Purpose**
Pin 1 (VSS)	GND	Ground
Pin 2 (VDD)	5V	LCD power
Pin 3 (V0)	GND or potentiometer	Contrast
Pin 4 (RS)	PB0	Register Select
Pin 5 (RW)	GND	Always Write
Pin 6 (EN)	PB1	Enable Strobe
Pin 11 (D4)	PB2	Data bit 4
Pin 12 (D5)	PB3	Data bit 5
Pin 13 (D6)	PB4	Data bit 6
Pin 14 (D7)	PB5	Data bit 7
Backlight A (LED+)	5V	*Backlight Anode*
Backlight K (LED–)	GND	*Backlight Cathode*

TABLE II: MFRC522 to STM32 SPI Interface Connections

MFRC522 Pin	Connects To STM32 Pin	Function
VCC	3.3V	Power (do NOT use 5V)
GND	GND	Ground
SDA / SS / NSS	PA4	Chip Select (CS)
SCK	PA5	SPI Clock
MOSI	PA7	Master Out Slave In
MISO	PA6	Master In Slave Out
RST	PB12	Reset pin
IRQ	*Not used*	(leave unconnected)

TABLE III: Button Interfacing with STM32 Microcontroller

Button	STM32 Pin	Wiring
CONFIRM button	PC13	PC13 $\rightarrow$ Button $\rightarrow$ GND
EXIT/HISTORY button	PC14	PC14 $\rightarrow$ Button $\rightarrow$ GND

C. MFRC522 RFID Reader Configuration

The first section outlines the connections for the **MFRC522 RFID Reader module**, the core component for RFID interaction. This module uses the **Serial Peripheral Interface (SPI)** protocol for fast, synchronous data transfer.

The use of PA4, PA5, PA6, and PA7 for SPI suggests utilizing the **SPI1 peripheral** of the STM32, which is critical for the high-speed data transfer needed for reliable tag reading.

D. 16x2 LCD (4-bit Mode) Configuration

This section maps the connections for a standard **16x2 Character Liquid Crystal Display (LCD)**, used for user feedback. The **4-bit mode** minimizes pin usage by utilizing only four data lines (D4-D7).

This configuration dedicates **PB1, PB2, and PB10-PB13** for the display, crucial for the user interface in the cashless system.

E. Auxiliary Modules and Debugging

This final section details the connections for essential auxiliary components, including alerts and development tools.

F. Buzzer

The **Buzzer** provides audible feedback for events (e.g., successful transaction, warnings).

G. UART Debug

The **Universal Asynchronous Receiver/Transmitter (UART)** is used for development and serial debugging.

H. I2C (Optional)

The **Inter-Integrated Circuit (I2C)** bus is reserved for future expansion, demonstrating a scalable design.

I. Conclusion

The pinout table illustrates an efficient embedded system design, utilizing a **master-slave SPI connection** for the RFID reader, a **pin-saving 4-bit LCD interface** for output, and dedicated **peripheral pins** for debugging and alerts. The choice of pins from both Port A (PA) and Port B (PB) ensures optimal use of the STM32's resources and peripheral mapping capabilities.

TABLE IV: Buzzer Connection to STM32

Buzzer Pin	STM32 Pin
+	PA8
-	GND

TABLE V: MFRC522 RFID Reader Connections

Signal	STM32 Pin	Protocol	Description
SDA / NSS	PA4	SPI	**SPI Chip Select (NSS)**
SCK	PA5	SPI	**SPI Clock (SCK)**
MOSI	PA7	SPI	**Master Out Slave In (MOSI)**
MISO	PA6	SPI	**Master In Slave Out (MISO)**
RST	PB0	GPIO	**RFID Reset Pin**

III. Commands and Status Codes

A. Status Codes (from `mfrc522.h`)

The header file also defines status codes used to indicate the result of an operation:

- `MI_OK`: `0`
- `MI_NOTAGERR`: `1`
- `MI_ERR`: `2`

B. LCD 16x2 Commands

- **Clear display:** Clears the entire display and sets the Display Data RAM (DDRAM) address to **0** in the address counter.
- **Return home:** Sets the **DDRAM address to 0** in the address counter. It also returns the display from being shifted to its original position. DDRAM contents remain unchanged.
- **Entry mode set:** Sets the cursor **move direction** (I/D) and specifies the **display shift** (S). These operations are performed during data write and read.
- **Display on/off control:** Sets the entire display (**D**) on/off, the cursor (**C**) on/off, and the **blinking** (**B**) of the cursor position character.
- **Cursor or display shift:** Moves the **cursor** or shifts the **entire display** without changing the DDRAM contents.
- **Function set:** Sets the interface **data length** (**DL**), the **number of display lines** (**N**), and the **character font** (**F**).
- **Set CGRAM address:** Sets the **Character Generator RAM (CGRAM) address**. CGRAM data is sent and received after this setting.
- **Set DDRAM address:** Sets the **Display Data RAM (DDRAM) address**. DDRAM data is sent and received after this setting.

IV. Core Functions

A. Functions from `lcd.h` (LCD Display Control)

These functions provide the main interface for controlling a character LCD module.

- **void lcd_init (void);**
  - **Purpose:** Initializes the LCD module for operation. This involves configuring the hardware interface (GPIO

TABLE VI: 16x2 LCD Connections (4-bit Mode)

Signal	STM32 Pin	Description
RS	PB1	**Register Select**
EN	PB2	**Enable Signal**
D4	PB10	**Data Bit 4**
D5	PB11	**Data Bit 5**
D6	PB12	**Data Bit 6**
D7	PB13	**Data Bit 7**

TABLE VII: Buzzer Connection

Signal	STM32 Pin	Description
BUZZ	PA8	**Alert / Beep Output:** Signals events to the user.

TABLE VIII: UART Debug Connection

Signal	STM32 Pin	Description
TX/RX	PA9 / PA10	**Serial Debugging**

TABLE IX: I2C Optional Connection

Signal	STM32 Pin	Description
SDA/SCL	PB7 / PB6	Integration of new I2C-based sensors or memory.

TABLE X: MFRC522 PCD and PICC Commands

Command Group	Command Define	Hex Value
PCD (Reader) Commands	PCD_IDLE	0x00
	PCD_AUTHENT	0x0E
	PCD_RECEIVE	0x08
	PCD_TRANSMIT	0x04
	PCD_TRANSCEIVE	0x0C
	PCD_RESETPHASE	0x0F
	PCD_CALCCRC	0x03
PICC (Tag) Commands	PICC_REQIDL	0x26
	PICC_REQALL	0x52
	PICC_ANTICOLL	0x93
	PICC_SElECTTAG	0x93
	PICC_AUTHENT1A	0x60
	PICC_AUTHENT1B	0x61
	PICC_READ	0x30
	PICC_WRITE	0xA0
	PICC_DECREMENT	0xC0
	PICC_INCREMENT	0xC1
	PICC_RESTORE	0xC2
	PICC_TRANSFER	0xB0
	PICC_HALT	0x50

TABLE XI: HD44780 LCD Controller Instruction Codes

Instruction	RS	R/W	8-bit Code (DB7-DB0)
Clear display	0	0	0000 0001
Return home	0	0	0000 001–
Entry mode set	0	0	0000 01I/D S
Display on/off control	0	0	0000 1D C B
Cursor or display shift	0	0	0001 S/C R/L –
Function set	0	0	0000 1DL N F –
Set CGRAM address	0	0	01 ACG ACG ACG ACG ACG ACG
Set DDRAM address	0	0	1 ADD ADD ADD ADD ADD ADD ADD

pins) and sending the necessary command sequence (like setting it to 4-bit mode, display on, cursor settings) to prepare the LCD for receiving data.

- **void lcd_clear(void);**
  - **Purpose:** Clears all characters from the LCD screen and resets the cursor position to the beginning of the first line (home position).
- **void lcd_print(const char *str);**
  - **Purpose:** Writes the specified **null-terminated C-string** (`str`) to the LCD starting at the current cursor position. The cursor automatically advances after each character is printed.
- **void lcd_print_line(uint8_t line, const char *str);**
  - **Purpose:** A utility function that first moves the cursor to the start of the specified **`line`** (e.g., line 0 or line 1) and then prints the string (`str`) from that position.

B. Functions from `mfrc522.h` (RFID Reader Control)

These functions are the high-level API for performing common tasks with the MFRC522 RFID reader, such as detecting and selecting tags.

- **void MFRC522_Init(void);**
  - **Purpose:** Initializes the MFRC522 chip. This typically includes performing a **software reset**, setting the default register values for the RFID communication protocol, and **turning on the antenna**.
- **uint8_t MFRC522_Request(uint8_t reqMode, uint8_t *TagType);**
  - **Purpose:** Searches for an RFID tag (PICC) within the reader's operating field.
  - **`reqMode`** specifies the type of request (e.g., PICC_REQIDL).
  - If a tag is found, the **`TagType`** (card type code) is returned via the pointer.
  - **Returns:** `MI_OK` on success, or an error code.
- **uint8_t MFRC522_Anticoll(uint8_t *serNum);**
  - **Purpose:** Executes the anti-collision protocol to read the unique **Serial Number (UID)** of a detected tag.
  - The UID is stored in the **`serNum`** array.
  - **Returns:** `MI_OK` on success.
- **uint8_t MFRC522_SelectTag(uint8_t *serNum);**
  - **Purpose:** Selects the tag with the given serial number (`serNum`) to transition it to the ACTIVE state, enabling subsequent operations.
  - **Returns:** The size of the tag's internal cascade level on success.
- **void MFRC522_Halt(void);**
  - **Purpose:** Commands the currently selected tag (PICC) to enter the **HALT** state, effectively deselecting it so it won't respond until a new MFRC522_Request() is performed.

C. Auxiliary/Low-Level MFRC522 Functions (Internal Use)

These functions are typically called by the high-level MFRC522 API and not directly by the main application:

- MFRC522_WriteRegister, MFRC522_ReadRegister: For direct SPI register access.
- MFRC522_SetBitMask, MFRC522_ClearBitMask: For modifying register bits.
- MFRC522_AntennaOn, MFRC522_AntennaOff: For RF field control.
- MFRC522_ToCard: The core function for transmitting and receiving data.

V. Workflow

A. Core Decision Point and System Reset

The flow begins at the START/RESET state, leading directly to the primary decision node: CARD TYPE?. This crucial step determines whether the tapped RFID card belongs to a vendor (initiating a purchase) or a student (managing their account).

- Vendor Tap: Follows the path for item selection and payment.
- Student Tap: Branches into options for balance check, adding funds, or viewing history.

A key feature of the system is the mechanism for returning to the initial state, ensuring continuous operation and security:

- Completion: All successful and unsuccessful transaction paths (like PAYMENT EXECUTED or DISPLAY WARNING) lead to a final RESET block.
- Timeout: The diagram notes that "No activity detected for 5 seconds: reset" will return the system to START/RESET.
- Manual Exit: The note "Exit button pushed: reset" also forces a return to the start.

A key feature of the system is the mechanism for returning to the initial state, ensuring continuous operation and security:

- **Completion:** All successful and unsuccessful transaction paths (like **PAYMENT EXECUTED** or **DISPLAY WARNING**) lead to a final **RESET** block.
- **Timeout:** The diagram notes that *"No activity detected for 5 seconds: reset"* will return the system to **START/RESET**.
- **Manual Exit:** The note *"Exit button pushed: reset"* also forces a return to the start.

B. Vendor Transaction Workflow (Purchase Path)

This path covers the steps required for a student to purchase an item from a vendor using their wallet.

- **DISPLAY ITEMS:** After a vendor tap, the system first shows the available products or services.
- **CYCLE ITEMS:** The vendor or student can then **cycle** through the displayed items, presumably using navigational controls.
- **ITEM SELECTED / PRICE DISPLAYED:** Once the desired item is chosen, this state is reached, and the corresponding price is clearly shown.
- **SUFFICIENT BALANCE?** This is the second critical decision node, where the system checks the student's current wallet balance against the item's price.

Decision Outcomes:

- **Yes (Sufficient Balance):** The flow proceeds to **confirm button**. Pushing the **confirm button** moves the system to the **PAYMENT EXECUTED** state. This successful completion leads to a **RESET**.
- **No (Insufficient Balance):** The flow proceeds to the **DISPLAY WARNING** state, informing the user that the transaction cannot be completed. This path also terminates at a **RESET**.

C. Student Account Management Workflow

If the tapped card is identified as a **student tap** at the **CARD TYPE?** node, the system presents the student with three distinct options to manage their digital wallet: **SHOW BALANCE**, **ADD** (funds), or **SHOW HISTORY**.

D. Check Balance

- **SHOW BALANCE:** The student taps on this option.
- **confirm button:** The student confirms the action.
- **BALANCE SHOWCASED:** The system displays the current wallet balance.
- **RESET:** The process concludes and resets.

E. Add Funds

- **ADD:** The student taps this option.
- **confirm button:** The student confirms the intention to add funds.
- **BALANCE ADDED:** This state represents the successful completion of the fund-adding process (e.g., after inserting cash or completing a digital transfer via a connected module, although the external funding mechanism is abstracted here).
- **RESET:** The process concludes and resets.

F. View Transaction History

- **SHOW HISTORY:** The student taps this option.
- **confirm button:** The student confirms the request.
- **HISTORY SHOWCASED:** The system displays the recent transaction history for the wallet.
- **RESET:** The process concludes and resets.

G. System Summary

This workflow is a clear, single-entry, multi-exit design, ensuring that every user interaction, whether for a purchase or account management, is properly initiated and concluded by a **RESET**. This structure guarantees that the terminal is always ready for the next user or vendor interaction. The use of clear decision nodes (**CARD TYPE?** and **SUFFICIENT BALANCE?**) ensures the system handles all necessary conditions for a secure and functional cashless payment environment.

VI. How SPI Interfaces the MRFC522

SPI is a **synchronous** serial data protocol that operates in a full-duplex manner (data can be sent and received simultaneously). It requires four main communication lines. The MRFC522 acts as the **Slave** device, and the host microcontroller (e.g., an STM32 or Arduino) acts as the **Master**.

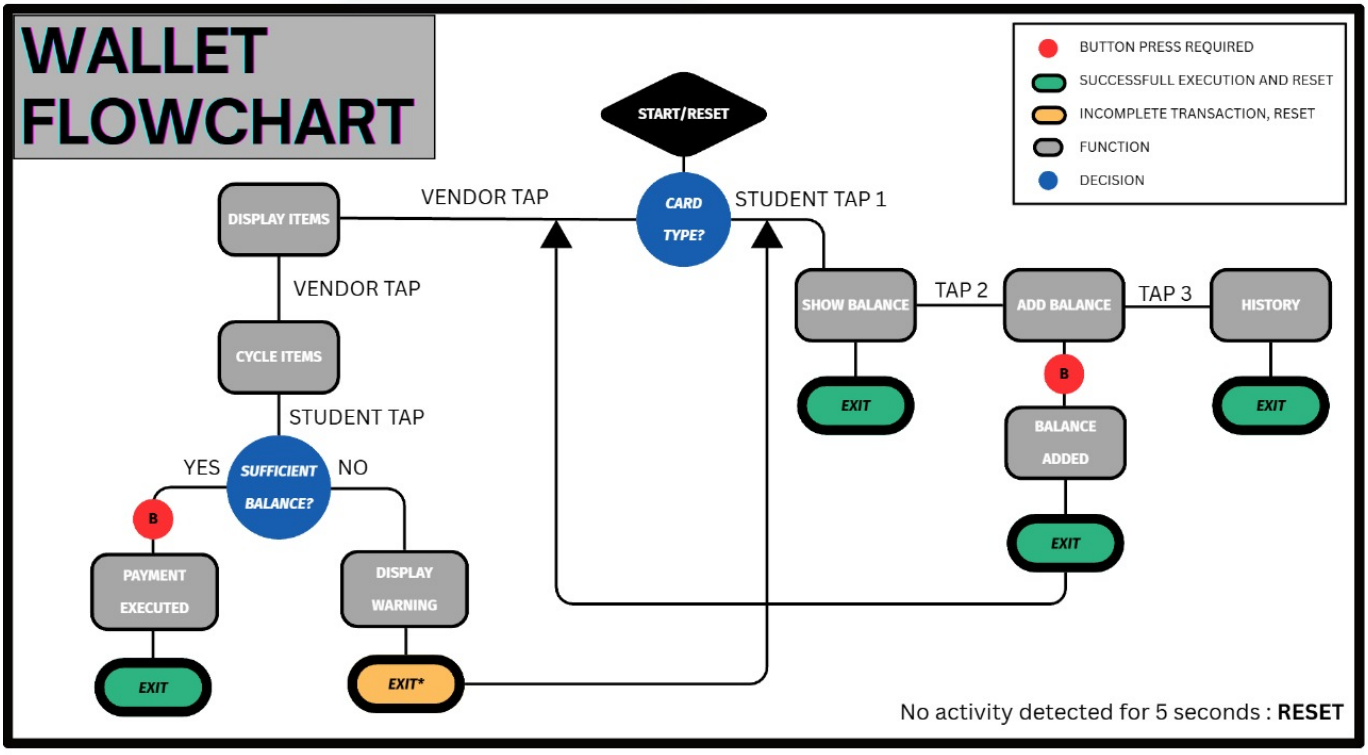

Fig. 1: Workflow Diagram

A. SPI Communication Lines

The interface involves the following dedicated pins:

Pin Name	Host (Master)	MRFC522 (Slave)
SCK	Serial Clock (O)	Serial Clock (I)
MOSI	Master Out, Slave In (O)	Slave In (I)
MISO	Master In, Slave Out (I)	Slave Out (O)
NSS (or CS)	Slave Select (O)	Slave Select (I)

B. Operation Summary

The data exchange follows these steps:

1) **Selection:** The host pulls the **NSS** pin of the MRFC522 **LOW** to begin a transaction.
2) **Clocking:** The host begins generating a clock signal on the **SCK** line.
3) **Data Exchange:** The host simultaneously shifts an 8-bit register address or data payload out on the **MOSI** line, while the MRFC522 shifts its response/status out on the **MISO** line.
4) **Deselection:** The host pulls the **NSS** pin **HIGH** to terminate the transaction.

VII. Why SPI is Preferred

SPI is the preferred choice over other serial protocols like I^2C or UART for the MRFC522 due to its key advantages:

- **High Speed:** SPI can operate at clock rates up to several MHz. This is critical for quickly processing RFID card data, which involves time-sensitive anti-collision and cryptographic routines.
- **Simple Protocol Overhead:** Unlike UART, there are no complex start/stop bits, and unlike I^2C, there is no need for address encoding/decoding within the data stream. Synchronization is handled entirely by the **SCK** line.
- **Full-Duplex Capability:** While the protocol might be used primarily for separate read and write operations, the ability to transfer data in both directions simultaneously can enhance overall data throughput.
- **Dedicated Slave Select (NSS):** The dedicated control line allows the host to manage multiple peripheral devices on the same SPI bus, selectively activating only the MRFC522 when needed.

In summary, SPI provides the **optimum balance of speed, simplicity, and reliability** necessary for controlling a high-performance peripheral like the MRFC522 RFID reader.

VIII. LCD Parallel Interface (4-bit Mode)

Most character LCDs are configured to operate in **4-bit mode** to conserve host microcontroller I/O pins, requiring a minimum of 6 control/data lines (plus power and ground). This mode relies on distinct **Control Pins** to manage the communication type and timing, and **Data Pins** to carry the information.

A. Pin Functions

The key interface pins are as follows:

Pin Name	Function (Protocol Role)
RS	**Register Select**: LOW for Command/Instruction register, HIGH for Data register
R/W	**Read/Write**: LOW for Write operation, HIGH for Read operation
EN	**High-to-Low pulse** tells the LCD driver to read the current state of the data bus and execute
D4 - D7	**Data Lines**: Used to transfer the 8-bit command or data in two separate 4-bit chunks (*nibbles*).

B. Comparison to SPI

While both interfaces use control signals to manage data flow, the mechanisms are distinct:

- **Type Selection:** In LCD interfacing, the operation type (Command or Data) is selected by the dedicated **RS** pin. In SPI, this information is typically encoded within the first byte of the data stream (register address).
- **Timing:** LCD timing is triggered by an asynchronous **EN** pulse, whereas SPI timing is dictated by the synchronous **SCK** (Serial Clock) generated by the host.

IX. The 4-Bit Interfacing Process

Since only four data lines (D4 – D7) are used, a single 8-bit command or character must be transmitted as two successive 4-bit nibbles.

1) **Setup Control:** The host sets the **RS** pin (LOW for command, HIGH for character data) and the **R/W** pin (usually LOW for write).
2) **Transmit Higher Nibble:** The host places the **upper 4 bits** of the 8-bit byte onto the data lines (D4 – D7).
3) **Latch Nibble 1:** The host sends a brief **High-to-Low pulse on the EN pin**. The LCD latches the first 4 bits.
4) **Transmit Lower Nibble:** The host places the **lower 4 bits** of the 8-bit byte onto the data lines (D4 – D7).
5) **Latch Nibble 2:** The host sends a second brief **High-to-Low pulse on the EN pin**. The LCD combines the two received nibbles to form the complete 8-bit instruction or character.

This method saves I/O pins but requires the host software to manage the two-step transmission process and ensure precise timing for the EN pulses.

X. Main Code

A. Definitions

Listing 1: Config

```
void SystemClock_Config(void);
static void MX_GPIO_Init(void);
static void MX_SPI1_Init(void);
static void MX_USART2_UART_Init(void);
void Error_Handler(void);

/* Peripheral handles */
SPI_HandleTypeDef hspi1;
UART_HandleTypeDef huart2;

/* route printf to UART2 */
int __io_putchar(int ch) {
    HAL_UART_Transmit(&huart2, (uint8_t *)&ch, 1,
        HAL_MAX_DELAY);
    return ch;
}

/* ---------------- CONFIG ---------------- */
#define MAX_STUDENTS    2
#define MAX_HISTORY     10
#define ITEMS_PER_SHOP  3
#define VENDOR_COUNT    2
#define LOW_BAL         20
#define INACTIVITY_MS   7000    /* general no-
    activity => ready */
#define STUDENT_MENU_TIMEOUT_MS 5000 /* student menu
     timeout (5s) */
#define VENDOR_TIMEOUT_MS 5000 /* vendor card
    selection timeout (5s) */
#define CONFIRM_TIMEOUT_MS 5000

/* Vendor UIDs (from you) */
static const uint8_t vendor_uids[VENDOR_COUNT][5] =
    {
    {0xB7,0xC8,0x64,0xB2,0xA9}, /* vendor 0 (canteen
        ) */
    {0xF7,0x88,0x60,0xB2,0xAD}  /* vendor 1 (
        stationery) */
};
static const int vendor_shop_map[VENDOR_COUNT] =
    {0,1};

/* Shops / Items / Prices */
enum { SHOP_CANTEEN=0, SHOP_STATIONERY=1 };
static const char *shop_names[] = {"CANTEEN","
    STATIONERY"};
static const char *items[2][ITEMS_PER_SHOP] = {
    {"SAMOSA","MAGGI","FRUITY"},
    {"PEN","NOTE","ERASR"}
};
static const int prices[2][ITEMS_PER_SHOP] = {
    {10,25,15},
    {5,20,3}
};

/* Students (UIDs provided) */
static uint8_t student_uids[MAX_STUDENTS][5] = {
    {0x67,0x5B,0x93,0xB2,0x1D},
    {0x82,0x82,0x5A,0x30,0x6A}
};
static const char *student_name[MAX_STUDENTS] = {"
    Saumya","Ravi"};
static const char *student_roll[MAX_STUDENTS] = {"
    R123","R124"};
static const char *student_snu[MAX_STUDENTS]  = {"
    SNU001","SNU002"};
static uint32_t student_bal[MAX_STUDENTS] =
    {200,150};

/* History */
typedef struct { char txt[48]; uint32_t t; } hist_t;
static hist_t shop_history[MAX_HISTORY];
static int shop_hist_count = 0;
static hist_t student_history[MAX_STUDENTS][
    MAX_HISTORY];
static int student_hist_count[MAX_STUDENTS] = {0};
```

B. Helpers

Listing 2: Helpers

```
static inline int uid_eq(const uint8_t *a,const
    uint8_t *b){
    for(int i=0;i<5;i++) if(a[i]!=b[i]) return 0;
    return 1;
}
static int vendor_index_for_uid(const uint8_t *u){
    for(int i=0;i<VENDOR_COUNT;i++) if(uid_eq(u,
        vendor_uids[i])) return i;
    return -1;
}
static int find_student(const uint8_t *u){
    for(int i=0;i<MAX_STUDENTS;i++) if(uid_eq(u,
        student_uids[i])) return i;
    return -1;
}
static void push_shop_history(const char *s){
    int idx = shop_hist_count % MAX_HISTORY;
    snprintf(shop_history[idx].txt, sizeof(
        shop_history[idx].txt), "%s", s);
    shop_history[idx].t = HAL_GetTick();
    shop_hist_count++;
}
static void push_student_history(int stu, const char
     *s){
    if(stu<0 || stu>=MAX_STUDENTS) return;
    int idx = student_hist_count[stu] % MAX_HISTORY;
    snprintf(student_history[stu][idx].txt, sizeof(
        student_history[stu][idx].txt), "%s", s);
    student_history[stu][idx].t = HAL_GetTick();
    student_hist_count[stu]++;
}

/* buzzer helpers (active buzzer) */
static void beep_pulse(uint16_t ms){
    HAL_GPIO_WritePin(GPIOA, GPIO_PIN_8,
        GPIO_PIN_SET);
    HAL_Delay(ms);
    HAL_GPIO_WritePin(GPIOA, GPIO_PIN_8,
        GPIO_PIN_RESET);
}
static void beep_detect(){ beep_pulse(40); }
static void beep_success(){ beep_pulse(180); }
static void beep_lowbal(){ beep_pulse(400); }
static void beep_error(){ beep_pulse(120); HAL_Delay
    (80); beep_pulse(120); }

/* LCD helpers */
static void show_ready(void){
    lcd_clear();
    lcd_print_line(1,"SNU RFID Wallet");
    lcd_print_line(2,"is ready");
}
static void show_shop_idle(void){ show_ready(); }
static void show_student_basic(int s){
    char l2[17];
    lcd_clear();
    lcd_print_line(1, student_name[s]);
    snprintf(l2, sizeof(l2), "Bal:%lu", (unsigned
        long)student_bal[s]);
    lcd_print_line(2, l2);
}
static void show_add_prompt(void){
    lcd_clear();
    lcd_print_line(1,"ADD +100 ?");
    lcd_print_line(2,"Press CONFIRM");
}
static void show_history_student(int s){
    int cnt = student_hist_count[s] > MAX_HISTORY ?
        MAX_HISTORY : student_hist_count[s];
    if(cnt == 0){
        lcd_clear(); lcd_print_line(1,"HISTORY:");
            lcd_print_line(2,"No records");
        HAL_Delay(1100); return;
    }
    int start = student_hist_count[s] - cnt;
    for(int i=0;i<cnt;i++){
        int idx = (start + i) % MAX_HISTORY;
        lcd_clear();
        lcd_print_line(1,"HISTORY:");
        char ln[17]; strncpy(ln, student_history[s][
            idx].txt, 16); ln[16]=0;
        lcd_print_line(2, ln);
        HAL_Delay(1200);
    }
    /* === ADDED: 2 second beep AFTER showing
        student's history === */
    beep_pulse(2000);
}
```

C. Main

Listing 3: Main

```
int main(void)
{
    HAL_Init();
    SystemClock_Config();
    MX_GPIO_Init();
    MX_SPI1_Init();
    MX_USART2_UART_Init();

    printf("UART OK\r\n");

    MFRC522_Init();
    lcd_init();
    lcd_clear();
    HAL_Delay(50);

    /* startup beep */
    beep_pulse(80);

    show_shop_idle();

    /* state */
    uint8_t uid[5];
    int card_present_prev = 0;
    uint32_t last_activity = HAL_GetTick();      /*
        any activity */
    uint32_t last_student_activity = 0;          /*
        student menu timestamp */
    uint32_t last_vendor_activity = 0;           /*
        vendor selection timestamp */

    int selectedShop = -1;
    int vendor_cycle_idx = -1;
    int selectedItem = -1;

    int menu_student = -1;
    int menu_state = 0; /* 0 balance,1 add,2 history
        ,3 exit */

    while (1)
    {
        int chk = MFRC522_Check(uid); /* MI_OK if
            present */
        int card_present = (chk == MI_OK) ? 1 : 0;
        int new_tap = (card_present && !
            card_present_prev);
        int removed = (!card_present &&
            card_present_prev);

        if (card_present) {
            last_activity = HAL_GetTick();
```

```
        if (menu_student >= 0)
            last_student_activity = HAL_GetTick
            ();
        if (selectedItem >= 0)
            last_vendor_activity = HAL_GetTick()
            ;
    }

    /* General inactivity -> main ready (7s) */
    if (!card_present && (HAL_GetTick() -
        last_activity >= INACTIVITY_MS)
        && menu_student < 0 && selectedItem < 0)
    {
        selectedItem = -1; selectedShop = -1;
            vendor_cycle_idx = -1;
        menu_student = -1; menu_state = 0;
        show_shop_idle();
        last_activity = HAL_GetTick();
    }

    /* Student-menu-specific timeout (5s) */
    if (menu_student >= 0) {
        if (HAL_GetTick() -
            last_student_activity >=
            STUDENT_MENU_TIMEOUT_MS) {
            menu_student = -1; menu_state = 0;
            show_shop_idle();
            last_activity = HAL_GetTick();
        }
    }

    /* Vendor selection timeout (5s): if vendor
        selected but no vendor activity, reset
        */
    if (selectedItem >= 0) {
        if (HAL_GetTick() - last_vendor_activity
             >= VENDOR_TIMEOUT_MS) {
            selectedItem = -1;
            selectedShop = -1;
            vendor_cycle_idx = -1;
            show_shop_idle();
            last_activity = HAL_GetTick();
        }
    }

    /* NEW TAP handling */
    if (new_tap)
    {
        last_activity = HAL_GetTick();
        beep_detect();

        /* vendor card? */
        int vidx = vendor_index_for_uid(uid);
        if (vidx >= 0)
        {
            selectedShop = vendor_shop_map[vidx
                ];
            if (vendor_cycle_idx < 0)
                vendor_cycle_idx = 0;
            else vendor_cycle_idx = (
                vendor_cycle_idx + 1) %
                ITEMS_PER_SHOP;
            selectedItem = vendor_cycle_idx;
            last_vendor_activity = HAL_GetTick()
                ;

            char b2[17];
            snprintf(b2, sizeof(b2), "%s %d",
                items[selectedShop][selectedItem
                ], prices[selectedShop][
                selectedItem]);
            lcd_clear(); lcd_print_line(1,
                shop_names[selectedShop]);
                lcd_print_line(2, b2);
            printf("Vendor: %s | Item: %s (%d)\r
                \n", shop_names[selectedShop],
                items[selectedShop][selectedItem
                ], prices[selectedShop][
                selectedItem]);
            HAL_Delay(400);
            card_present_prev = card_present;
            continue;
        }

        /* student card? */
        int stu = find_student(uid);
        if (stu < 0)
        {
            /* unknown card during payment
                attempt -> cancel vendor
                selection and go idle */
            if (selectedItem >= 0 &&
                selectedShop >= 0)
            {
                lcd_clear(); lcd_print_line(1,"
                    UNKNOWN CARD");
                    lcd_print_line(2,"Payment
                    Cancel");
                printf("Unregistered attempted
                    payment UID: %02X %02X %02X
                    %02X %02X\r\n", uid[0],uid
                    [1],uid[2],uid[3],uid[4]);
                beep_error();
                selectedItem = -1; selectedShop
                    = -1; vendor_cycle_idx = -1;
                show_shop_idle();
                HAL_Delay(900);
                while (MFRC522_Check(uid) ==
                    MI_OK) HAL_Delay(80);
                card_present_prev = 0;
                    last_activity = HAL_GetTick
                    ();
                continue;
            }

            /* otherwise unknown */
            lcd_clear(); lcd_print_line(1,"
                UNKNOWN CARD"); lcd_print_line
                (2,"Contact Admin");
            beep_error(); HAL_Delay(900);
            card_present_prev = card_present;
                last_activity = HAL_GetTick();
            continue;
        }

        /* if vendor selected => payment flow */
        if (selectedItem >= 0 && selectedShop >=
             0)
        {
            int price = prices[selectedShop][
                selectedItem];
            char l2[17]; snprintf(l2, sizeof(l2)
                , "Pay %d Rs", price);
            lcd_clear(); lcd_print_line(1, items
                [selectedShop][selectedItem]);
                lcd_print_line(2, l2);
            printf("Await CONFIRM for %s to pay
                %d\n", student_name[stu], price)
                ;

            uint32_t t0 = HAL_GetTick(); int
                confirmed = 0;
            while (HAL_GetTick() - t0 <
                CONFIRM_TIMEOUT_MS)
            {
                if (HAL_GPIO_ReadPin(GPIOC,
                    GPIO_PIN_13) ==
                    GPIO_PIN_RESET) { confirmed
```

```
            = 1; break; }
        HAL_Delay(40);
    }

    if (!confirmed)
    {
        lcd_clear(); lcd_print_line(1,"
            PAY CANCEL"); lcd_print_line
            (2,"Timeout");
        HAL_Delay(1200);
        selectedItem = -1; selectedShop
            = -1; vendor_cycle_idx = -1;
        show_shop_idle();
        card_present_prev = card_present
            ; last_activity =
            HAL_GetTick();
        continue;
    }

    /* do payment */
    if (student_bal[stu] >= (uint32_t)
        price)
    {
        student_bal[stu] -= price;
        char rec[128];
        snprintf(rec, sizeof(rec), "%s
            paid %s %d Rs Bal:%lu (%s,%s
            )",
                 student_name[stu],
                     items[selectedShop
                     ][selectedItem],
                     price,
                 (unsigned long)
                     student_bal[stu],
                     student_roll[stu],
                     student_snu[stu]);
        push_shop_history(rec);
            push_student_history(stu,
            rec);

        lcd_clear(); lcd_print_line(1,"
            PAID OK"); lcd_print_line(2,
             items[selectedShop][
            selectedItem]);
        printf("RECEIPT: %s\r\n", rec);
        beep_success();
        HAL_Delay(1800);

        if (student_bal[stu] < LOW_BAL)
        {
            beep_lowbal();
            lcd_clear(); lcd_print_line
                (1,"LOW BALANCE");
                lcd_print_line(2,"Please
                 Topup");
            HAL_Delay(2000);
        }
    }
    else
    {
        lcd_clear(); lcd_print_line(1,"
            INSUFFICIENT");
            lcd_print_line(2,"TOP-UP REQ
            ");
        beep_error();
        HAL_Delay(1800);
    }

    selectedItem = -1; selectedShop =
        -1; vendor_cycle_idx = -1;
    show_shop_idle();
    while (MFRC522_Check(uid) == MI_OK)
        HAL_Delay(80);
```

```
            card_present_prev = 0; last_activity
                 = HAL_GetTick();
            continue;
        }

        /* student menu cycling */
        if (menu_student != stu)
        {
            menu_student = stu; menu_state = 0;
            show_student_basic(stu);
            last_student_activity = HAL_GetTick
                ();
            card_present_prev = card_present;
                last_activity = HAL_GetTick();
            HAL_Delay(300);
            continue;
        }
        else
        {
            menu_state = (menu_state + 1) % 4;
            if (menu_state == 0)
                show_student_basic(stu);
            else if (menu_state == 1)
                show_add_prompt();
            else if (menu_state == 2)
                show_history_student(stu);
            else { menu_student = -1; menu_state
                 = 0; show_shop_idle(); }
            last_student_activity = HAL_GetTick
                ();
            card_present_prev = card_present;
                last_activity = HAL_GetTick();
            HAL_Delay(300);
            continue;
        }
    } /* end new_tap */

    /* Non-blocking confirm for ADD page */
    if (menu_student >= 0 && menu_state == 1)
    {
        if (HAL_GPIO_ReadPin(GPIOC, GPIO_PIN_13)
             == GPIO_PIN_RESET)
        {
            HAL_Delay(80);
            if (HAL_GPIO_ReadPin(GPIOC,
                GPIO_PIN_13) == GPIO_PIN_RESET)
            {
                int stu = menu_student;
                student_bal[stu] += 100;
                char rec[128];
                snprintf(rec, sizeof(rec), "
                    Topup +100 by %s Bal:%lu (%s
                    ,%s)",
                         student_name[stu], (
                             unsigned long)
                             student_bal[stu],
                             student_roll[stu],
                             student_snu[stu]);
                push_shop_history(rec);
                    push_student_history(stu,
                    rec);

                lcd_clear(); lcd_print_line(1,"
                    TOPUP DONE"); lcd_print_line
                    (2,"+100 Added");
                beep_success();
                HAL_Delay(1400);

                menu_state = 2;
                while (MFRC522_Check(uid) ==
                    MI_OK) HAL_Delay(80);
                card_present_prev = 0;
                    last_activity = HAL_GetTick
                    (); last_student_activity =
```

```
                        HAL_GetTick();
                    continue;
                }
            }
        }

        /* Exit (PC14) resets UI */
        if (HAL_GPIO_ReadPin(GPIOC, GPIO_PIN_14) ==
            GPIO_PIN_RESET)
        {
            selectedItem = -1; selectedShop = -1;
                vendor_cycle_idx = -1;
            menu_student = -1; menu_state = 0;
            show_shop_idle();
            HAL_Delay(300); last_activity =
                HAL_GetTick();
        }

        card_present_prev = card_present;
        HAL_Delay(60);
    } /* main loop */

    return 0;
}
```

REFERENCES

[1] Official STM32F303RET6 Data Sheet
[2] RC522 Data Sheet
[3] LCD 16x2 Data Sheet